

Data Structures & Systems Design

SE Intern Assessment · Himasree D

PROBLEM 1 — LRU CACHE IMPLEMENTATION

Approach & Data Structure Rationale

The core requirement here is $O(1)$ for both get and put. Achieving that forces a specific combination: a hash map for instant key lookup, and a doubly linked list to track usage order without having to scan through anything.

The hash map maps each key directly to its node in the list. The doubly linked list orders nodes from most-recently-used at the head to least-recently-used at the tail. When a key is accessed, its node is unlinked and moved to the front — both operations are $O(1)$ because we have direct pointers. When the cache is full, the node just before the tail sentinel is the LRU candidate and can be removed in $O(1)$ as well.

Sentinel head and tail nodes are used to eliminate edge-case checks for empty lists or single-element lists. They hold no real data — they exist purely to make `_remove` and `_insert_at_front` unconditionally safe.

A plain dict alone would give $O(1)$ lookup but no way to evict in $O(1)$, since finding the oldest key would require a full scan. A singly linked list would make removal $O(n)$ because you cannot walk backwards to relink. The doubly linked list and dict together solve both problems.

Implementation

```
class Node:
    def __init__(self, key: int, value: int):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None

class LRUCache:
    """
    O(1) get and put via hash map + doubly linked list.
    List is arranged most-recent (head) → least-recent (tail).
    Sentinel head/tail nodes make insert/remove unconditionally safe.
    """
    def __init__(self, capacity: int):
        if capacity <= 0:
            raise ValueError("Capacity must be a positive integer.")
        self.capacity = capacity
        self.cache: dict[int, Node] = {}

        # Sentinel nodes – never store real data
        self.head = Node(0, 0)
        self.tail = Node(0, 0)
        self.head.next = self.tail
        self.tail.prev = self.head
```

```

def _remove(self, node: Node) -> None:
    node.prev.next = node.next
    node.next.prev = node.prev

def _insert_at_front(self, node: Node) -> None:
    node.next = self.head.next
    node.prev = self.head
    self.head.next.prev = node
    self.head.next = node

def get(self, key: int) -> int:
    if key not in self.cache:
        return -1
    node = self.cache[key]
    self._remove(node)
    self._insert_at_front(node)
    return node.value

def put(self, key: int, value: int) -> None:
    if key in self.cache:
        self._remove(self.cache[key])
    node = Node(key, value)
    self.cache[key] = node
    self._insert_at_front(node)
    if len(self.cache) > self.capacity:
        lru = self.tail.prev
        self._remove(lru)
        del self.cache[lru.key]

```

PROBLEM 2 — EVENT SCHEDULER

can_attend_all — Approach

Sort events by start time, then sweep through them tracking the furthest end time seen so far. If any event starts before that running maximum end time, there is a genuine overlap and the function returns False.

A common mistake here is comparing each event against only its immediate predecessor's end time — that fails when an early event has a long span covering several later ones. Tracking `max_end` as a running variable catches those cases correctly.

min_rooms_required — Approach

The key insight is that you do not need to simulate room assignments. You only need to know how many meetings are running concurrently at any given moment — that peak is the answer.

Sort all start times and all end times into two separate arrays. Use two pointers, one over starts and one over ends. At each step: if the next meeting begins before any currently-running meeting has ended, a new room is needed. If a meeting has already ended by the time the next one starts, that room is freed. The maximum value of the running room count is the answer.

This works because sorted end times tell you the earliest possible moment a room becomes free. We never need to know which specific room frees up — only that some room does.

Implementation

```
def can_attend_all(events: list[tuple[int, int]]) -> bool:
    """
    Returns True if a single attendee can attend all events.
    Adjacent events (end == start) are not considered overlapping.
    Time: O(n log n)    Space: O(1)
    """
    if not events:
        return True
    events_sorted = sorted(events, key=lambda x: x[0])
    max_end = events_sorted[0][1]
    for i in range(1, len(events_sorted)):
        start, end = events_sorted[i]
        if start < max_end:
            return False
        max_end = max(max_end, end)
    return True

def min_rooms_required(events: list[tuple[int, int]]) -> int:
    """
    Returns the minimum rooms needed to host all events simultaneously.
    Time: O(n log n)    Space: O(n)
    """
    if not events:
        return 0
    starts = sorted(e[0] for e in events)
    ends = sorted(e[1] for e in events)
    rooms = max_rooms = 0
    s = e = 0
    while s < len(events):
        if starts[s] < ends[e]:
            rooms += 1
            max_rooms = max(max_rooms, rooms)
            s += 1
        else:
            rooms -= 1
            e += 1
    return max_rooms
```

FINAL DISCUSSION & ANALYSIS

Time & Space Complexity

Function	Time	Space
LRUCache.get()	O(1)	O(1)
LRUCache.put()	O(1)	O(1) amortized

LRUCache – total storage	—	$O(\text{capacity})$
can_attend_all()	$O(n \log n)$	$O(1)$
min_rooms_required()	$O(n \log n)$	$O(n)$
assign_rooms()	$O(n \log n)$	$O(n)$

Trade-offs: Hash Map + Doubly Linked List

Each structure compensates for the other's weakness.

A hash map provides $O(1)$ access to any node by key, but it has no concept of ordering — it cannot tell you which entry was used longest ago without a scan.

A doubly linked list maintains a strict usage order and allows $O(1)$ node removal and insertion because every node holds pointers to both its neighbours. But without the hash map, finding a specific node in the list requires $O(n)$ traversal.

Together: the hash map is the index into the list. You look up the node in $O(1)$, then reposition it in $O(1)$. Neither structure alone achieves both goals. A singly linked list would not work here because removing a node requires repointing its predecessor — without a back pointer you cannot do that in $O(1)$.

Future Proofing: Assigning Named Rooms

The two-pointer approach in `min_rooms_required` intentionally discards which room frees up — it only counts. To assign names, you need to track the state of each room.

The right structure is a min-heap keyed on event end time, paired with a pool of available room names. When a new event arrives, pop all rooms from the heap whose end time falls at or before the event's start — those rooms are free and go back into the pool. Then assign a room from the pool (or create a new named room if the pool is empty), and push the new event's end time and assigned room name onto the heap.

This runs in $O(n \log n)$ and returns a mapping from each event to its room name, while reusing rooms correctly.

assign_rooms — Implementation

```
import heapq

def assign_rooms(
    events: list[tuple[int, int]]
) -> list[tuple[tuple[int, int], str]]:
    """
    Assigns a named room to each event.
    Reuses rooms when their previous meeting has ended.
    Time:  $O(n \log n)$    Space:  $O(n)$ 
    """
    if not events:
        return []

    indexed = sorted(enumerate(events), key=lambda x: x[1][0])
    room_counter = 0
    available: list[str] = []
```

```

active: list[tuple[int, str]] = [] # min-heap: (end_time, room_name)
result: list = [None] * len(events)

for orig_idx, (start, end) in indexed:
    # Free rooms whose meetings have ended
    while active and active[0][0] <= start:
        _, freed = heapq.heappop(active)
        available.append(freed)

    # Assign a room
    if available:
        room = available.pop()
    else:
        room_counter += 1
        room = f"Room {room_counter}"

    heapq.heappush(active, (end, room))
    result[orig_idx] = (events[orig_idx], room)

return result

```

Concurrency: Making LRU Cache Thread-Safe

The current implementation is not safe under concurrent access. Two threads calling `get()` and `put()` simultaneously can corrupt the linked list — for example, both reading the same LRU node before either removes it, leaving a dangling pointer.

The straightforward fix is wrapping both methods with a `threading.Lock`. Only one thread can hold the lock at a time, so reads and writes are serialized.

A subtlety worth noting: `get()` appears read-only, but it is not — it moves the accessed node to the front of the list. That mutation means a shared read lock is not safe here without additional design work. A plain lock is correct.

In higher-throughput systems, the single-lock approach becomes a bottleneck. The standard solution is lock striping: partition the cache into N independent shards, each with its own lock. Keys are routed to a shard via `hash(key) % N`. Threads operating on different shards proceed in parallel, and contention only occurs when two threads hit the same shard.

Thread-Safe Implementation

```

import threading

class ThreadSafeLRUCache(LRUCache):
    """
    Thread-safe LRU cache via a single mutex.
    get() is not truly read-only — it moves nodes — so a plain
    lock is required. For higher throughput, consider lock striping:
    split into N shards, route by hash(key) % N.
    """

    def __init__(self, capacity: int):
        super().__init__(capacity)
        self._lock = threading.Lock()

```

```
def get(self, key: int) -> int:
    with self._lock:
        return super().get(key)

def put(self, key: int, value: int) -> None:
    with self._lock:
        return super().put(key, value)
```